

Chương 1: Cơ sở lý thuyết và giới thiệu đề tài

1.1 Tổng quan về gian lận tín dụng:

1.1.1. Khái niệm gian lận thẻ tín dụng

Gian lận thẻ tín dụng là hình thức gian lận sử dụng công nghệ cao để đánh cắp thông tin thẻ tín dụng (Visa, MasterCard, ATM...) của người sử dụng thuộc về lĩnh vực tài chính, ngân hàng. Gian lận thẻ tín dụng là việc sử dụng trái phép thẻ ghi nợ hoặc thẻ tín dụng để mua hàng hoặc rút tiền mặt. Năm 2021, đã có 389.845 báo cáo về gian lận thẻ tín dụng tại Hoa Kỳ, và theo Ủy ban Thương mại Liên bang, đây là loại gian lận danh tính phổ biến nhất ảnh hưởng đến những người trong độ tuổi 20-39.

Gian lận thẻ tín dụng xảy ra khi tội phạm đánh cắp thông tin thẻ tín dụng của người khác và sử dụng thông tin đó để trục lợi cá nhân. Theo truyền thống, gian lận thẻ tín dụng xảy ra khi chủ thẻ bị đánh cắp thẻ vật lý. Tuy nhiên, với tình hình gian lận thẻ tín dụng hiện đại, kẻ gian ngày càng có khả năng lấy được thông tin thẻ tín dụng của nạn nhân, chứ không phải thẻ vật lý.

1.1.2. Các hình thức gian lận thẻ tín dụng

Các hình thức gian lận thẻ tín dụng thường thuộc một trong ba loại chính: gian lận đăng ký thẻ (application fraud) chiếm đoạt tài khoản (account takeover) và Gian lận giao dịch (Transaction Fraud).

Gian lận đăng ký thẻ (application fraud) là việc mở trái phép tài khoản thẻ tín dụng bằng tên của người khác. Hình thức gian lận này có thể xảy ra nếu thủ phạm thu thập đủ thông tin cá nhân của nạn nhân để hoàn thành toàn bộ đơn đăng ký thẻ tín dụng hoặc có khả năng tạo ra các tài liệu giả mạo tinh vi (trông như thật). Khi đơn đăng ký được xử lý thành công, thủ phạm sẽ sử dụng thẻ tín dụng mới để rút một lượng tiền mặt lớn, để lại khoản nợ cho người bị đánh cắp danh tính phải chi trả. Các chiêu trò gian lận đăng ký thẻ

rất nghiêm trọng vì, mặc dù nạn nhân không chịu trách nhiệm cho khoản nợ gian lận đó, họ có thể không phát hiện ra vụ gian lận cho đến khi nó đã ảnh hưởng xấu đến điểm tín dụng của họ hoặc sau khi họ đã lỡ thanh toán các khoản nợ đó. Vì việc đăng ký thẻ tín dụng yêu cầu kiểm tra tín dụng, việc thường xuyên yêu cầu báo cáo tín dụng hàng năm từ một trong ba văn phòng tín dụng có thể giúp nạn nhân của gian lận đăng ký thẻ phát hiện ra hành vi gian lận từ sớm.

Chiếm đoạt tài khoản (account takeover) thường liên quan đến việc chiếm quyền kiểm soát trái phép một tài khoản thẻ tín dụng hiện có, một thủ đoạn mà theo đó thủ phạm thu thập đủ thông tin cá nhân về nạn nhân để thay đổi địa chỉ thanh toán của tài khoản. Sau đó, thủ phạm báo thẻ bị mất hoặc bị đánh cắp để nhận được thẻ mới và thực hiện các giao dịch mua sắm gian lận bằng thẻ đó.

Giao dịch gian lận là hành vi trái phép hoặc bất hợp pháp liên quan đến việc sử dụng các công cụ thanh toán hoặc hệ thống tài chính, thường nhằm mục đích lấy tiền, hàng hóa hoặc dịch vụ mà không có sự đồng ý hoặc ủy quyền hợp lệ từ chủ tài khoản. Loại giao dịch này thường liên quan đến hành vi trộm cắp danh tính, thông tin thanh toán bị đánh cắp hoặc lừa đảo và nhằm mục đích gây thiệt hại tài chính cho chủ tài khoản, doanh nghiệp hoặc tổ chức tài chính. Giao dịch gian lận bao gồm hai loại là gian lận thẻ không xuất trình (CNP) và gian lận thẻ vật lý.

Gian lận thẻ không xuất trình (CNP) là một hình thức gian lận thẻ tín dụng, trong đó tội phạm thực hiện giao dịch bất hợp pháp bằng thông tin thẻ tín dụng bị đánh cắp. Nó có thể xảy ra thông qua giao dịch trực tuyến, điện thoại hoặc thư, và chúng ta cũng có thể gọi đây là gian lận "mua hàng từ xa". Đúng như tên gọi "gian lận thẻ không xuất trình", thẻ tín dụng (hoặc chủ thẻ)

không cần phải có mặt trong quá trình giao dịch. Một khi kẻ gian có đủ dữ liệu về chủ thẻ, chúng có thể thực hiện hành vi gian lận thẻ không xuất trình bất cứ lúc nào và ở bất cứ đâu, khiến việc phát hiện và bắt giữ trở nên khó khăn.

Gian lận Thẻ vật lý (Card-Present - CP) là hình thức gian lận truyền thống khi kẻ gian có được thẻ vật lý của nạn nhân (do bị đánh cắp) hoặc một bản sao của thẻ. Hình thức này đang trở nên ít phổ biến hơn nhờ sự ra đời của chip và mã PIN. Một phương pháp phổ biến được sử dụng để thực hiện gian lận này được gọi là "skimming" (sao chép thông tin thẻ). Các chiêu trò skimming xảy ra khi nhân viên của doanh nghiệp truy cập bất hợp pháp thông tin thẻ tín dụng của khách hàng. Sau đó, những nhân viên này bán thông tin cho những kẻ trộm danh tính hoặc tự mình chiếm đoạt danh tính của nạn nhân.

Những tiến bộ công nghệ cũng đã tạo ra nhiều con đường cho gian lận thẻ tín dụng. Với sự gia tăng của mua sắm trực tuyến, thủ phạm không còn cần thẻ vật lý để thực hiện giao dịch mua hàng trái phép. Ngoài ra, các cơ sở dữ liệu điện tử chứa dữ liệu thẻ tín dụng có thể bị tin tặc tấn công hoặc tự gặp sự cố, làm lộ thông tin thẻ tín dụng của khách hàng. Các vụ tấn công cơ sở dữ liệu điện tử này khiến tính bảo mật của nhiều tài khoản gặp rủi ro cùng một lúc.

Chính sự bùng nổ về số lượng giao dịch trực tuyến (CNP) và tính tinh vi của các phương thức tấn công dữ liệu đã khiến các hệ thống phát hiện gian lận truyền thống (dựa trên luật) trở nên không còn hiệu quả. Chúng không thể xử lý khối lượng dữ liệu khổng lồ hoặc thích ứng đủ nhanh với các chiêu thức mới. Điều này đã tạo ra một nhu cầu cấp thiết cho các giải pháp thông minh hơn, mở đường cho việc ứng dụng Trí tuệ nhân tạo (AI) và Học máy.

1.2. Trí tuệ nhân tạo trong phát hiện gian lận

1.2.1. Hiện trạng:

Các phương pháp phát hiện gian lận truyền thống thường không đáp ứng triệt để trong việc phòng tránh các hình thức gian lận mới ngày càng tinh vi. Do đó, sử dụng trí tuệ nhân tạo (AI) đang là một phương pháp hữu hiệu để xử lý các vấn đề này thông qua khả năng phân tích vượt trội dựa trên dữ liệu lớn, giúp tăng cường và bổ sung đáng kể các chiến lược phòng chống gian lận của ngân hàng. AI có thể xác định các mẫu, phát hiện sự bất thường và dự đoán các hoạt động gian lận với độ chính xác và tốc độ cao hơn các phương pháp truyền thống. Các tổ chức tài chính có thể sử dụng AI để giám sát các giao dịch theo thời gian thực, phân tích lượng dữ liệu khổng lồ và tự động hóa phản ứng trước các mối đe dọa tiềm ẩn.

AI đã thay đổi cách thức giao dịch, phân tích rủi ro và phát hiện gian lận, tạo ra các hệ thống tài chính an toàn và hiệu quả hơn. Điều này đặc biệt quan trọng trong bối cảnh thị trường tài chính toàn cầu ngày càng phức tạp. Các ngân hàng lớn như JPMorgan Chase sử dụng học máy để dự đoán rủi ro tín dụng và đánh giá khả năng vỡ nợ. Hơn nữa, các mô hình AI có thể phân tích hành vi bất thường trong giao dịch để phát hiện gian lận nhanh chóng và chính xác, góp phần bảo vệ hệ thống tài chính toàn cầu.

Hệ thống AI có thể giám sát các giao dịch trong thời gian thực, sử dụng thuật toán học máy để xác định các mẫu đáng ngờ và gắn cờ các giao dịch có khả năng gian lận. Bên cạnh đó, nhiều ngân hàng đã phát triển các giải pháp chấm điểm rủi ro gian lận. Thông qua AI, ngân hàng có thể chấm điểm rủi ro cho các giao dịch dựa trên nhiều yếu tố khác nhau đồng thời phân tích dữ liệu giao dịch lịch sử để dự đoán và ngăn chặn các hoạt động gian lận trong tương lai. Bài học điển hình JPMorgan Chase đã triển khai các hệ thống phát hiện gian lận dựa trên AI để giám sát các giao dịch thẻ tín dụng. Bằng cách phân tích mô hình giao dịch và hành vi của khách hàng, hệ thống AI có thể xác

định các giao dịch gian lận với độ chính xác cao. Ngân hàng báo cáo đã giảm đáng kể các trường hợp dương tính giả và tốc độ phát hiện gian lận được cải thiện.

PayPal sử dụng AI để phân tích lượng lớn dữ liệu giao dịch trong thời gian thực. Hệ thống AI xác định và gắn cờ các giao dịch đáng ngờ, cho phép PayPal ngăn chặn các hoạt động gian lận trước khi chúng gây ra tổn thất đáng kể. Việc triển khai AI đã giúp giảm đáng kể tổn thất tài chính liên quan đến gian lận và cải thiện niềm tin của khách hàng.

1.2.2. Ưu điểm:

Tốc độ và hiệu quả: AI có thể xử lý lượng dữ liệu khổng lồ với tốc độ cực nhanh (lightning speed), xác định các mẫu và hoạt động gian lận trong thời gian thực. Các phương pháp thủ công truyền thống đơn giản là không thể sánh kịp.

Phát hiện chính xác: Các mô hình Học máy, thông qua học sâu và mạng nơ-ron, có thể học hỏi và cải thiện theo thời gian, dẫn đến việc phát hiện gian lận với độ chính xác cao.

Phòng chống gian lận chủ động: Hệ thống AI có thể dự đoán các hoạt động gian lận tiềm ẩn, cho phép doanh nghiệp thực hiện các biện pháp phòng ngừa trước khi gian lận xảy ra.

Giảm thiểu lỗi do con người: AI loại bỏ rủi ro về lỗi của con người thường liên quan đến các phương pháp phát hiện gian lận truyền thống, làm tăng hiệu quả tổng thể của quy trình.

Tiết kiệm chi phí: Bằng cách phát hiện gian lận nhanh chóng và chính xác, doanh nghiệp có thể giảm đáng kể các tổn thất về tiền bạc liên quan đến gian lận.

Khả năng thích ứng: AI và ML có thể thích ứng với các loại gian lận mới bằng cách học hỏi từ các mẫu dữ liệu, chứng tỏ sự mạnh mẽ trong một bối cảnh kỹ thuật số không ngừng phát triển.

Khả năng mở rộng: Hệ thống AI có thể dễ dàng xử lý khối lượng dữ liệu ngày càng tăng, khiến chúng phù hợp cho việc mở rộng kinh doanh và tăng trưởng dữ liệu trong tương lai.

1.2.3. Thách thức:

Rủi ro về bảo mật và quyền riêng tư: Các ngân hàng nắm giữ một lượng lớn dữ liệu nhạy cảm của khách hàng. Việc tích hợp các mô hình AI, đặc biệt là những mô hình từ bên thứ ba, có thể tạo ra những lỗ hổng bảo mật mới. Tin tặc có thể tấn công vào hệ thống AI để đánh cắp thông tin hoặc thao túng các quyết định tài chính. Hơn nữa, việc sử dụng dữ liệu cá nhân để huấn luyện AI phải tuân thủ nghiêm ngặt các quy định về quyền riêng tư như Quy định Bảo vệ Dữ liệu Chung GDPR ở châu Âu hay các luật tương tự ở mỗi quốc gia. Bất kỳ sự vi phạm nào cũng có thể dẫn đến các khoản phạt khổng lồ.

Rủi ro về pháp lý và tuân thủ: Khung pháp lý cho AI vẫn đang trong giai đoạn hình thành, tạo ra một "vùng xám" cho các ngân hàng. Các cơ quan quản lý như Cục Bảo vệ Tài chính Người tiêu dùng (CFPB) tại Hoa Kỳ đã đưa ra cảnh báo rằng các tổ chức phải chịu trách nhiệm hoàn toàn cho các quyết định của thuật toán, bất kể chúng được phát triển nội bộ hay bởi một nhà cung cấp bên ngoài. Ngân hàng cần đảm bảo rằng hệ thống AI của họ minh bạch, có thể giải thích được và tuân thủ mọi quy định hiện hành.

Rủi ro từ "ảo giác" của AI: Đây là một vấn đề đặc trưng của AI tạo sinh. Các mô hình ngôn ngữ lớn đôi khi có thể tạo ra những thông tin hoàn toàn sai lệch nhưng lại được trình bày một cách rất thuyết phục. Hãy tưởng tượng một trợ lý ảo AI tư vấn cho khách hàng về một sản phẩm tài chính không tồn tại, hoặc một hệ thống phân tích thị trường đưa ra dự báo dựa trên dữ liệu "ảo". Hậu quả có thể rất tai hại, từ việc làm suy giảm lòng tin của khách hàng đến việc gây ra những tổn thất tài chính trực tiếp.

1.3 Các phương pháp học máy được sử dụng:

1.3.1 Các khái niệm cơ bản:

Máy học là thuật ngữ chung để chỉ việc máy tính học từ dữ liệu. Đây là một nhánh của Trí tuệ nhân tạo (AI), trong đó các thuật toán được sử dụng để thực hiện một nhiệm vụ cụ thể mà không cần lập trình rõ ràng – thay vào đó, chúng nhận diện các mẫu trong dữ liệu và đưa ra dự đoán dựa trên những gì đã học khi có dữ liệu mới. Máy học (ML) là một cách hiệu quả để tự động hóa các nhiệm vụ phức tạp, vượt xa hơn so với tự động hóa dựa trên quy tắc. Máy học (Machine Learning – ML) đã thu hút được nhiều sự chú ý trong những năm gần đây nhờ việc ứng dụng rộng rãi trong nhiều lĩnh vực. Từ phát hiện gian lận thẻ tín dụng đến quảng cáo nhắm mục tiêu trên mạng xã hội, ML đang được sử dụng cho những nhiệm vụ trước đây do con người thực hiện nhưng giờ có thể tự động hóa thông qua các thuật toán dựa trên cơ sở dữ liệu lớn.

Có ba cách phổ biến nhất mà máy móc có thể học: Học có giám sát (Supervised learning), Học không giám sát (Unsupervised learning) và Học tăng cường (Reinforcement learning)

Học có giám sát (Supervised learning) sử dụng dữ liệu đã được gán nhãn hoặc đánh dấu để huấn luyện các mô hình Máy học (ML). Các thuật toán có thể được đào tạo để phân loại dữ liệu chính xác hoặc dự đoán kết quả. Do đó, học có giám sát cho phép các doanh nghiệp giải quyết các vấn đề thực tế ở quy mô lớn, chẳng hạn như tách thư rác khỏi email. Học có giám sát thu thập dữ liệu từ kinh nghiệm trước đó hoặc tạo ra kết quả dữ liệu từ sự kiện đó. Nó giúp tối ưu hóa các yêu cầu về hiệu suất dựa trên kinh nghiệm trước đó và giải quyết nhiều vấn đề tính toán thực tế khác nhau

Học không giám sát (Unsupervised learning) đánh giá và phân nhóm dữ liệu không được gán nhãn. Các thuật toán này tự động phát hiện các mẫu ẩn hoặc nhóm dữ liệu. So với học có giám sát, các thuật toán học không giám sát

có thể xử lý các vấn đề phức tạp hơn. Học không giám sát cho phép các công ty khám phá dữ liệu, giúp họ phát hiện các mẫu nhanh hơn so với quan sát bằng con người. Học không giám sát tìm ra tất cả các mẫu chưa được phát hiện trước đó trong dữ liệu và hỗ trợ trong việc phát hiện các đặc điểm hữu ích cho việc phân loại.

Học tăng cường (Reinforcement learning) huấn luyện các mô hình máy tính để đưa ra quyết định bằng cách đặt trí tuệ nhân tạo (AI) vào một tình huống giống như trò chơi. Trong học tăng cường, thuật toán học qua phương pháp thử và sai (trial and error) bằng cách sử dụng phản hồi từ các hành động của nó. Mô hình Học máy (ML) nhận được phản hồi tích cực và tiêu cực về hành vi của nó và học từ chính kinh nghiệm của nó để tối đa hóa độ chính xác của kết quả đầu ra.

1.3.2 Phương pháp học máy phổ biến trong phát hiện gian lận:

Cây quyết định (Decision Tree) là một thuật toán học có giám sát phi tham số, được sử dụng cho cả bài toán phân loại và hồi quy. Nó có cấu trúc cây phân cấp, bao gồm một nút gốc, các nhánh, các nút trong và các nút lá. Học cây quyết định sử dụng chiến lược chia để trị bằng cách thực hiện tìm kiếm tham lam để xác định các điểm phân chia tối ưu trong cây. Quá trình phân chia này sau đó được lặp lại theo cách đệ quy từ trên xuống cho đến khi tất cả, hoặc phần lớn, các bản ghi được phân loại theo các nhãn lớp cụ thể.

Rừng ngẫu nhiên (Random Forest) là phương pháp mà một bộ phân loại sử dụng nhiều cây quyết định trên các tập con ngẫu nhiên của dữ liệu đầu vào để cải thiện độ chính xác dự đoán. Quá trình thực hiện bao gồm việc tạo tập dữ liệu mới, xây dựng cây quyết định, lặp lại quá trình này cho đủ số cây cần xây dựng, và sau đó, đưa ra dự đoán bằng cách gán nhãn dựa trên phiếu bầu của đa số cây. Random Forest được đánh giá cao với khả năng xử lý bài toán hồi quy và phân loại, đồng thời cung cấp thông tin về mức độ quan trọng của các thuộc tính. Tuy nhiên, nhược điểm của phương pháp này là thời gian xử lý dữ liệu và yêu cầu nhiều tài nguyên lưu trữ.

Hồi quy logistic (Logistic Regression) là một phương pháp phân tích dữ liệu sử dụng các phép toán để xác định mối quan hệ giữa các biến số (yếu tố dữ liệu). Dựa trên mối quan hệ đã tìm thấy, kỹ thuật này có thể dự đoán giá trị của một biến số dựa trên (các) biến số còn lại. Kết quả dự đoán thường là một giá trị rời rạc (discrete value), ví dụ: “Có” hoặc “Không”, “True” hoặc “False”.

Gradient Boosting là một thuật toán học máy mạnh mẽ theo hướng học tập tổ hợp (ensemble learning), cho phép tạo ra các dự đoán chính xác bằng cách kết hợp nhiều cây quyết định (decision trees) thành một mô hình duy nhất. Thuật toán này được giới thiệu bởi Jerome Friedman, hoạt động dựa trên nguyên tắc mỗi mô hình cơ sở sẽ học từ sai sót của mô hình trước đó. Qua từng vòng lặp, các mô hình mới dần điều chỉnh lỗi và cải thiện khả năng dự đoán. Nhờ khả năng nhận diện các mẫu phức tạp trong dữ liệu, Gradient Boosting đặc biệt hiệu quả trong các nhiệm vụ dự đoán.

XGBoost (Extreme Gradient Boosting) là một giải thuật được base trên gradient boosting, tuy nhiên kèm theo đó là những cải tiến to lớn về mặt tối ưu thuật toán, về sự kết hợp hoàn hảo giữa sức mạnh phần mềm và phần cứng, giúp đạt được những kết quả vượt trội cả về thời gian training cũng như bộ nhớ sử dụng.

1.4. Các chỉ số đánh giá mô hình:

Confusion matrix là một phương pháp đánh giá kết quả của những bài toán phân loại với việc xem xét cả những chỉ số về độ chính xác và độ bao quát của các dự đoán cho từng lớp. Một confusion matrix gồm 4 chỉ số sau đối với mỗi lớp phân loại:

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Để giải thích rõ 4 chỉ số này, ta sẽ áp dụng trực tiếp vào bài toán phát hiện gian lận thẻ tín dụng. Trong bài toán này, ta có 2 lớp: lớp "Gian lận" được xem là Positive (vì đây là đối tượng chúng ta cần tìm và phát hiện) và lớp "Hợp lệ" được xem là Negative.

TP (True Positive) - Bắt đúng: Số lượng dự đoán chính xác. Là khi mô hình dự đoán đúng một giao dịch thực sự là Gian lận.

TN (True Negative) - Tha đúng: Số lượng dự đoán chính xác. Là khi mô hình dự đoán đúng một giao dịch thực sự là Hợp lệ.

FP (False Positive - Lỗi loại I) - Bắt nhầm: Số lượng các dự đoán sai lệch. Là khi mô hình dự đoán một giao dịch là Gian lận (Positive), nhưng thực tế giao dịch đó hoàn toàn Hợp lệ (Negative). Gây phiền toái, làm gián đoạn trải nghiệm của khách hàng hợp lệ (ví dụ: thẻ bị khóa nhầm khi đang mua sắm).

FN (False Negative - Lỗi loại II) - Bỏ sót: Số lượng các dự đoán sai lệch. Là khi mô hình dự đoán một giao dịch là Hợp lệ (Negative), nhưng thực tế giao dịch đó lại là Gian lận (Positive). Đây là loại lỗi nguy hiểm nhất, vì nó khiến ngân hàng và khách hàng bị mất tiền.

Từ 4 chỉ số này, ta có 2 con số để đánh giá mức độ tin cậy của một mô hình:

Precision: Trong tất cả các dự đoán Positive được đưa ra, bao nhiêu dự đoán là chính xác? Đại diện cho mức độ "tin cậy" trong các lần mô hình báo động, Trong tất cả các giao dịch mà mô hình nói là Gian lận (TP + FP), có bao nhiêu phần trăm thực sự là gian lận (TP). Tối đa hóa Precision để giảm thiểu việc Bất nhảm (FP), tức là giảm thiểu việc làm phiền khách hàng hợp lệ.

$$\textbf{Precision} = \frac{\textbf{True Positive}}{\textbf{True Positive} + \textbf{False Positive}}$$

Recall : Trong tất cả các trường hợp Positive, bao nhiêu trường hợp đã được dự đoán chính xác. Đại diện cho khả năng của mô hình trong việc phát hiện (bắt) các vụ gian lận thực sự. Trong tất cả các giao dịch thực sự là Gian lận (TP + FN), mô hình của bạn đã Bắt đúng (TP) được bao nhiêu phần trăm. Tối đa hóa Recall để giảm thiểu việc Bỏ sót (FN), tức là giảm thiểu số tiền bị mất.

$$\textbf{Recall} = \frac{\textbf{True Positive}}{\textbf{True Positive} + \textbf{False Negative}}$$

Để đánh giá độ tin cậy chung của mô hình, người ta đã kết hợp 2 chỉ số Precision và Recall thành một chỉ số duy nhất: F-score, được tính theo công thức:

Một mô hình có chỉ số F-score cao chỉ khi cả 2 chỉ số Precision và Recall đều cao. Một trong 2 chỉ số này thấp đều sẽ kéo điểm F-score xuống. Trường hợp xấu nhất khi 1 trong hai chỉ số Precision và Recall bằng 0 sẽ kéo điểm F-score về 0. Trường hợp tốt nhất khi cả điểm chỉ số đều đạt giá trị bằng 1, khi đó điểm F-score sẽ là 1.

Area Under Curve(AUC) là một phép đo tổng hợp về hiệu suất của phân loại nhị phân trên tất cả các giá trị ngưỡng có thể có. Để hiểu rõ hơn về metric này, chúng ta sẽ tìm hiểu về một khái niệm cơ sở trước, đó là ROC Curve. ROC Curve (The receiver operating characteristic curve) là một đường cong biểu diễn hiệu suất phân loại của một mô hình phân loại tại các ngưỡng threshold. Về cơ bản, nó hiển thị True Positive Rate (TPR) so với False Positive Rate (FPR) đối với các giá trị ngưỡng khác nhau. Các giá trị TPR, FPR được tính như sau:

$$\text{TPR} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

$$\text{FPR} = \frac{\text{False Positive}}{\text{True Negative} + \text{False Negative}}$$

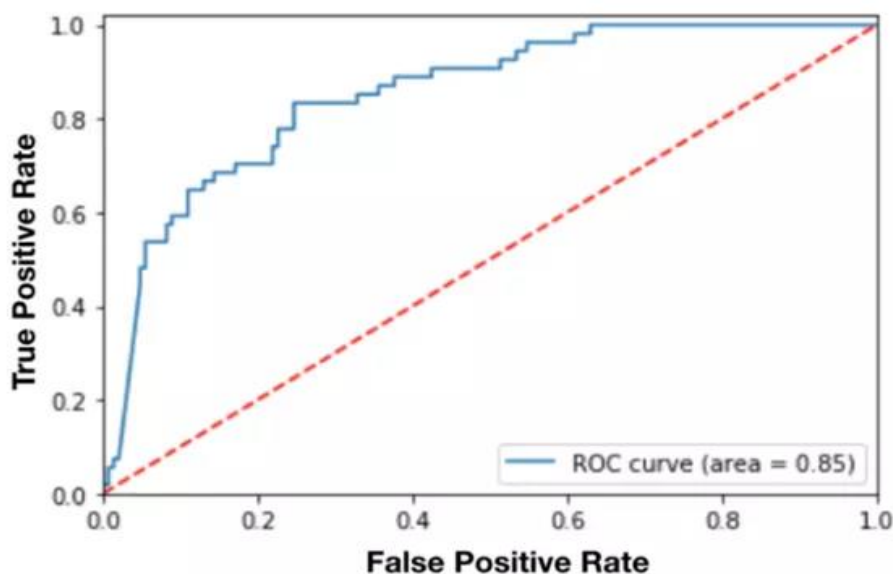
Ví dụ mô hình của bạn dự đoán giá trị xác suất (rằng một giao dịch là gian lận) cho 4 giao dịch lần lượt là [0.45, 0.6, 0.7, 0.3]. Tùy vào giá trị ngưỡng (threshold) mà chúng ta sẽ có các nhãn dự đoán khác nhau:

Ngưỡng là 0.5: Giao dịch 2 và 3 được dự đoán là Gian lận (vì $0.6 > 0.5$ và $0.7 > 0.5$).

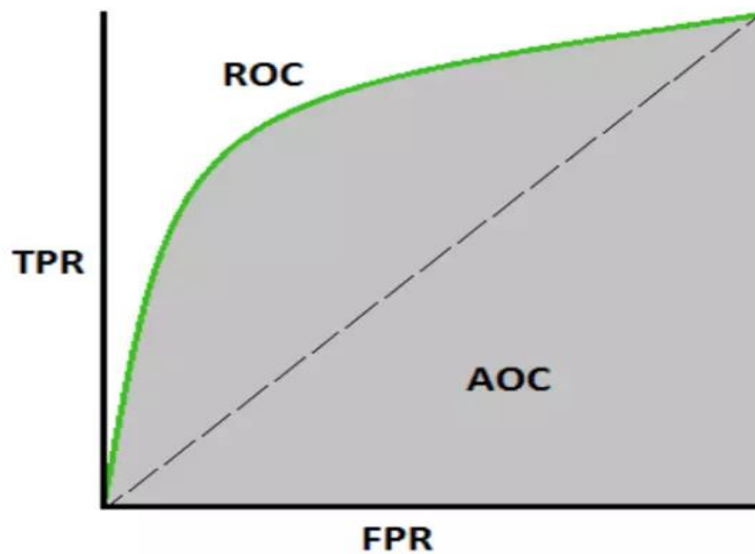
Ngưỡng là 0.25: Tất cả các giao dịch đều được dự đoán là Gian lận (vì tất cả xác suất đều > 0.25).

Ngưỡng là 0.8: Tất cả các giao dịch đều được dự đoán là Hợp lệ (vì không có xác suất nào > 0.8).

Có thể thấy với các ngưỡng khác nhau, chúng ta sẽ có kết quả dự đoán nhãn khác nhau, kéo theo các giá trị như precision hay recall cũng sẽ khác nhau. ROC tìm ra TPR và FPR ứng với các giá trị ngưỡng khác nhau và vẽ biểu đồ để dễ dàng quan sát TPR so với FPR. Ví dụ dưới đây là một đường cong ROC.



AUC là chỉ số được tính toán dựa trên đường cong ROC nhằm đánh giá khả năng phân loại của mô hình tốt như thế nào. Phần diện tích nằm dưới đường cong ROC và trên trục hoành chính là AUC, có giá trị nằm trong khoảng $[0, 1]$.



Khi diện tích này càng lớn, đường cong này sẽ dần tiệm cận với đường thẳng $y=1$ tương đương với khả năng phân loại của mô hình càng tốt. Còn khi đường cong ROC nằm sát với đường chéo đi qua hai điểm $(0, 0)$ và $(1, 1)$, mô hình sẽ tương đương với một phân loại ngẫu nhiên.

Chương 2: Giới thiệu bài toán và bộ dữ liệu

2.1. Giới thiệu bài toán phát hiện gian lận thẻ tín dụng

Bài toán phát hiện gian lận tín dụng là một trong những vấn đề quan trọng hàng đầu trong lĩnh vực tài chính - ngân hàng hiện nay. Phòng chống gian lận không chỉ giúp giảm thiểu tổn thất tài chính mà còn góp phần bảo vệ uy tín và lòng tin của khách hàng đối với ngân hàng. Theo báo cáo của Ủy ban Thương mại Liên bang Mỹ (FTC), chỉ riêng năm 2021 đã ghi nhận khoảng 2,8 triệu trường hợp gian lận, gây thiệt hại lên đến 5,8 tỷ USD. Đáng chú ý, đối với các ngân hàng, mỗi đô la bị mất do gian lận sẽ kéo theo khoảng 4 đô la chi phí bổ sung để khắc phục hậu quả, bao gồm chi phí điều tra, xử lý và thiệt hại về danh tiếng.

Đặc điểm chung của các hành vi gian lận là rất khó phát hiện, thường ẩn trong khối lượng dữ liệu giao dịch khổng lồ, và chỉ chiếm tỷ lệ rất nhỏ so với tổng số giao dịch hợp lệ. Hơn nữa, hành vi gian lận thay đổi liên tục để né tránh các biện pháp phát hiện, đòi hỏi hệ thống giám sát phải thông minh, linh hoạt và có khả năng tự học. Nếu không được phát hiện kịp thời, gian lận có thể dẫn đến những hậu quả nghiêm trọng như thiệt hại tài chính trực tiếp, ảnh hưởng uy tín tổ chức, mất lòng tin khách hàng và gia tăng chi phí kiểm soát rủi ro.

2.2. Phát hiện gian lận tín dụng dựa trên AI

Phát hiện gian lận tín dụng là quá trình ứng dụng các phương pháp thống kê, học máy hoặc trí tuệ nhân tạo để nhận diện và ngăn chặn các hành vi gian lận tài chính. Quá trình này dựa trên việc phân tích dữ liệu giao dịch, hành vi người dùng, hồ sơ vay và các đặc điểm liên quan để xác định xem một giao dịch hoặc khách hàng có hợp lệ hay không. Mục tiêu chính của hệ thống phát hiện gian lận là phân loại chính xác các giao dịch, giảm thiểu thiệt hại tài chính, đồng thời duy trì sự cân bằng giữa độ nhạy và độ chính xác nhằm hạn chế các trường hợp cảnh báo sai. Khi được triển khai hiệu quả, hệ thống không chỉ giúp ngân hàng giảm rủi ro và chi phí điều tra mà còn nâng cao trải nghiệm và niềm tin của khách hàng.

Trước đây, các ngân hàng thường sử dụng hệ thống phát hiện gian lận dựa trên quy tắc (rule-based), chẳng hạn như quy định “giao dịch trên 50 triệu đồng trong đêm thì cảnh báo”. Tuy nhiên, phương pháp này dần bộc lộ hạn chế khi hành vi gian lận ngày càng tinh vi và thay đổi liên tục. Hệ thống rule-based

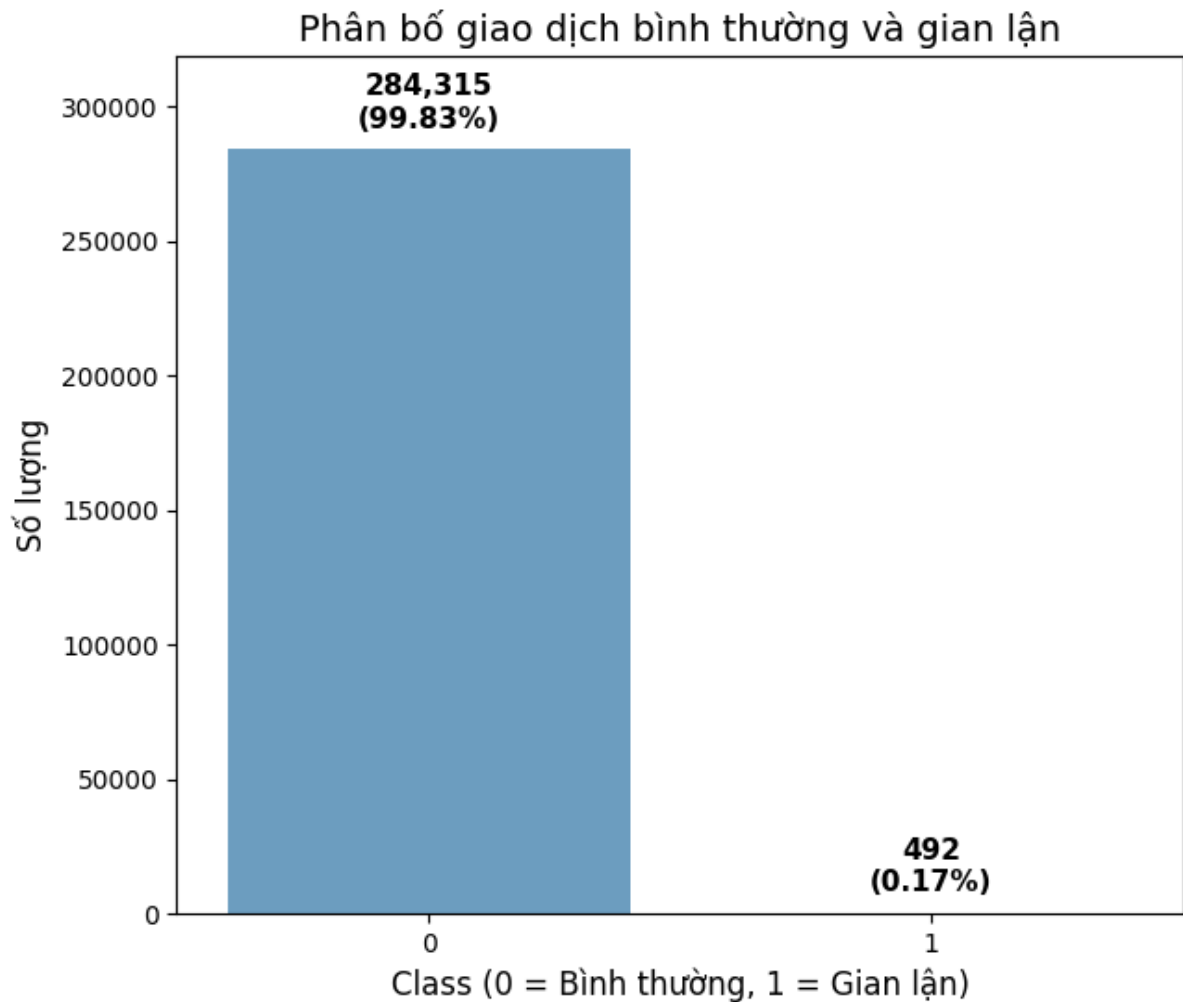
có độ chính xác cao trong các kịch bản cố định, nhưng khó mở rộng và dễ tạo ra tỷ lệ cảnh báo sai cao. Việc cập nhật quy tắc cũng tốn nhiều nhân lực và thời gian. Ngược lại, hệ thống dựa trên trí tuệ nhân tạo (AI) và học máy (ML) có khả năng tự học từ dữ liệu, thích ứng linh hoạt với các mẫu gian lận mới và xử lý dữ liệu lớn với tốc độ cao. AI có thể phát hiện các mối quan hệ phi tuyến, phân tích ngữ cảnh phức tạp và giảm thiểu cảnh báo sai, giúp phát hiện các hành vi bất thường mà con người khó nhận biết. Do đó, AI đang trở thành xu hướng tất yếu trong công tác phát hiện và phòng chống gian lận tài chính.

Trí tuệ nhân tạo cho phép giám sát giao dịch theo thời gian thực, phát hiện bất thường chỉ trong vài mili-giây, đồng thời kết hợp nhiều nguồn dữ liệu như thông tin giao dịch, thiết bị, vị trí địa lý, tần suất giao dịch và hành vi người dùng. Các mô hình học sâu (Deep Learning) có thể tự động trích xuất đặc trưng ẩn mà con người không thể thấy, từ đó dự đoán chính xác hơn nguy cơ gian lận. Ngoài ra, AI còn có khả năng học trực tuyến (online learning), giúp mô hình cập nhật và thích nghi liên tục với các kiểu gian lận mới.

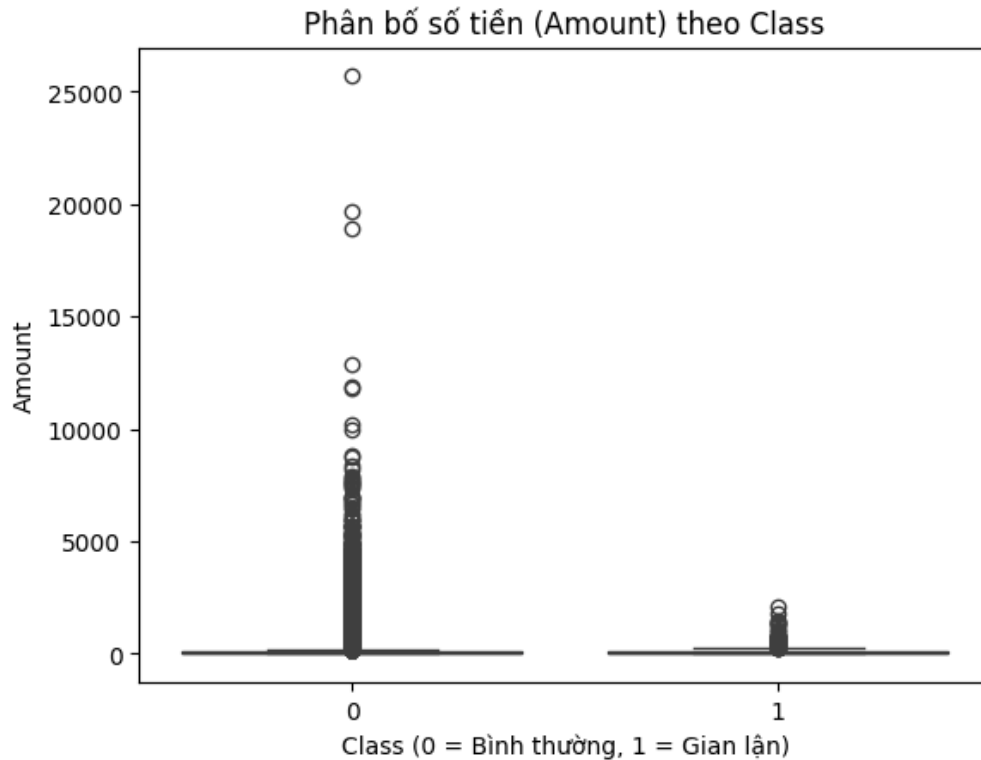
2.3. Giới thiệu bộ dữ liệu

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	0
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	0
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	123.50	0
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99	0

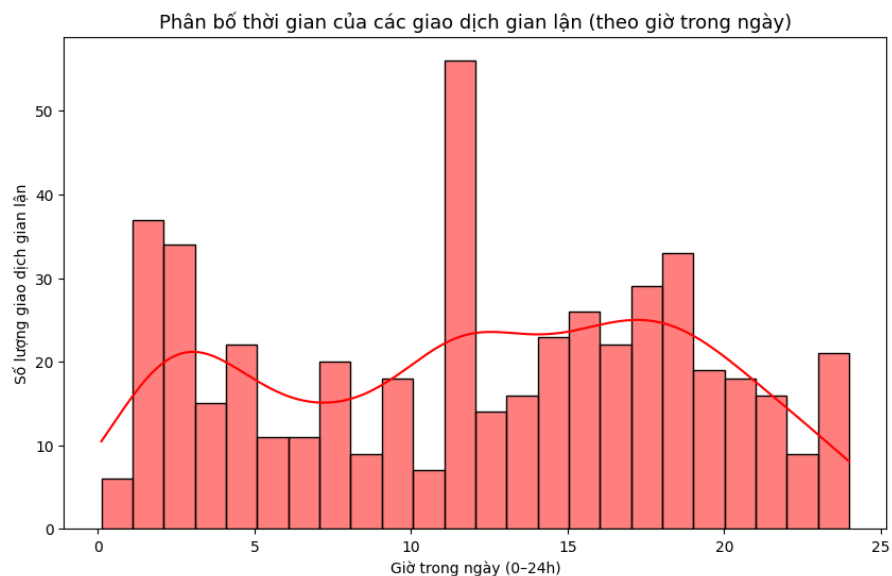
Trong nghiên cứu mô phỏng, bộ dữ liệu được sử dụng là bộ dữ liệu giao dịch thẻ tín dụng của khách hàng châu Âu năm 2013, đã được ẩn danh bằng phương pháp PCA để bảo vệ thông tin cá nhân. Bộ dữ liệu gồm 284.807 giao dịch với 31 thuộc tính, trong đó có 30 biến đầu vào (V1-V28, Time, Amount) và một biến nhãn (Class) cho biết giao dịch là hợp lệ hay gian lận. Chỉ có 492 giao dịch gian lận, chiếm khoảng 0,17% tổng số giao dịch, thể hiện tính mất cân bằng dữ liệu rất lớn. Điều này có nghĩa là nếu không xử lý phù hợp, mô hình học máy có thể học lệch theo xu hướng “mọi giao dịch đều bình thường” và bỏ sót các trường hợp gian lận thực sự.

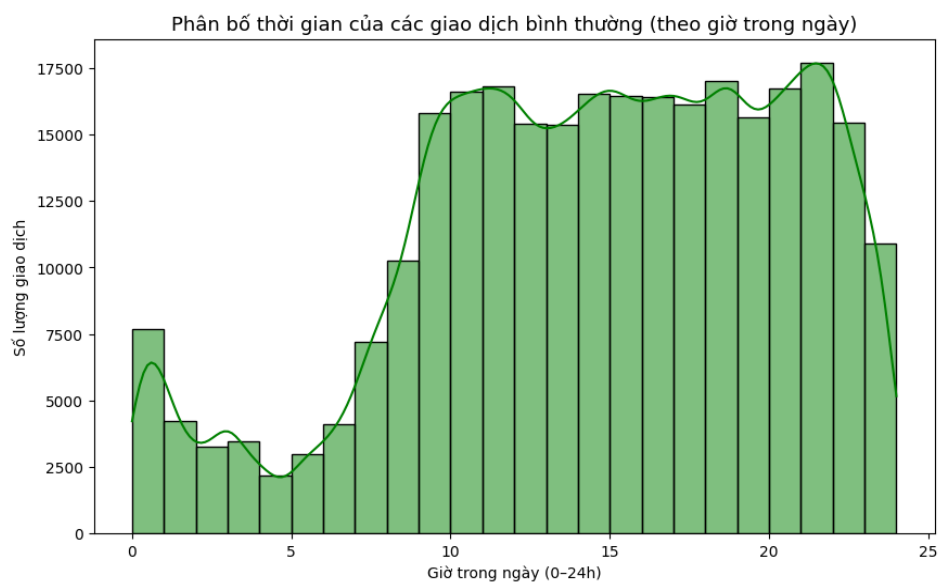


Khi phân tích phân bố giá trị giao dịch (Amount), ta thấy rằng các giao dịch hợp lệ trải rộng ở nhiều mức giá trị khác nhau, trong khi các giao dịch gian lận thường tập trung ở giá trị nhỏ. Trong thực tế, kẻ gian thường thử nghiệm với các giao dịch nhỏ để kiểm tra tính hợp lệ của thẻ trước khi thực hiện các giao dịch lớn. Vì vậy, giá trị giao dịch đơn lẻ không đủ để phát hiện gian lận. Mô hình cần xem xét kết hợp nhiều đặc trưng khác nhau như thời gian giao dịch, vị trí địa lý, tần suất giao dịch và thiết bị sử dụng. Ví dụ, nếu một người thường chỉ giao dịch vào ban ngày nhưng đột ngột có nhiều giao dịch nhỏ vào lúc 3 giờ sáng từ địa chỉ IP lạ, mô hình sẽ gắn cờ cảnh báo dù số tiền không lớn.



Khi phân tích yếu tố thời gian, sự khác biệt rõ rệt được nhận thấy giữa hai nhóm giao dịch. Các giao dịch hợp lệ thường phân bố đều trong khung giờ hành chính, phản ánh hành vi tự nhiên của người dùng. Trong khi đó, các giao dịch gian lận lại có xu hướng tập trung vào các thời điểm nhạy cảm như giờ nghỉ trưa, đêm khuya hoặc cuối ngày – khi người dùng ít giám sát tài khoản và hệ thống giám sát hoạt động kém hiệu quả hơn. Điều này cho thấy thời gian là một đặc trưng quan trọng cần được mô hình chú trọng để tăng độ chính xác trong phát hiện gian lận.





Từ toàn bộ quá trình phân tích có thể thấy rằng phát hiện gian lận tín dụng dựa trên AI không chỉ giúp ngân hàng nâng cao khả năng bảo mật, mà còn cải thiện hiệu quả vận hành và trải nghiệm khách hàng. Trong bối cảnh chuyển đổi số của ngành tài chính - ngân hàng, việc ứng dụng AI vào phát hiện gian lận không còn là lựa chọn mà đã trở thành yêu cầu tất yếu để thích ứng với môi trường rủi ro ngày càng phức tạp và biến động.

Chương 3: Xây dựng mô hình

3.1. Thử nghiệm bài toán phát hiện gian lận thẻ tín dụng với 3 mô hình: **Logistics Regression, Random Forest, XGBoost.**

3.1.1. Hồi quy Logistic

=== Huấn luyện Logistic Regression ===

```
model = LogisticRegression(  
    solver='liblinear',    # phù hợp cho dữ liệu nhị phân  
    penalty='l2',          # regularization chuẩn  
    C=1.0,                 # mức regularization (có thể thử 0.5, 2.0)  
    random_state=42,  
    max_iter=2000  
)
```

Phân tích chi tiết các tham số của mô hình Logistic Regression

Trong nghiên cứu này, mô hình hồi quy logistic (Logistic Regression) được lựa chọn nhằm giải quyết bài toán phát hiện gian lận trong các giao dịch tài chính. Đây là một mô hình học máy tuyến tính cổ điển nhưng mạnh mẽ, được sử dụng phổ biến trong các bài toán phân loại nhị phân. Mô hình ước lượng xác suất xảy ra của biến phụ thuộc thông qua hàm logistic (sigmoid), từ đó chuyển đổi đầu ra tuyến tính thành giá trị trong khoảng từ 0 đến 1. Việc lựa chọn Logistic Regression không chỉ giúp mô hình dự đoán xác suất xảy ra hành vi gian lận, mà còn cho phép diễn giải mức độ ảnh hưởng của từng biến độc lập, giúp nhà phân tích hiểu rõ hơn cơ chế của hiện tượng gian lận.

Tham số $solver='liblinear'$ – Bộ giải thuật tối ưu hóa:

Tham số $solver$ quy định thuật toán được sử dụng để tìm bộ trọng số tối ưu của mô hình trong quá trình huấn luyện. Đối với Logistic Regression, việc tối ưu hóa hàm mất mát (log loss) là một quá trình quan trọng, quyết định tốc độ hội tụ và độ ổn định của mô hình. Trong nghiên cứu này, bộ giải *liblinear* được lựa chọn vì nó sử dụng phương pháp *coordinate descent*, phù hợp cho các bài toán phân loại nhị phân và có quy mô dữ liệu vừa hoặc nhỏ.

Bộ giải này đặc biệt hiệu quả khi sử dụng với điều chuẩn L1 hoặc L2 và mang lại tốc độ hội tụ nhanh, ổn định. So với các thuật toán như *lbfgs* hay *saga* (thường dùng cho dữ liệu lớn và đa lớp), *liblinear* có ưu điểm là độ chính xác cao và tính ổn định tốt trong các bài toán nhỏ đến trung bình. Việc chọn *liblinear* đảm bảo mô hình huấn luyện trơn tru, tránh lỗi hội tụ và đạt được nghiệm tối ưu phù hợp với bản chất của dữ liệu phát hiện gian lận.

Tham số $penalty='l2'$ – Hình phạt điều chuẩn:

Tham số *penalty* xác định loại điều chuẩn (regularization) được áp dụng trong quá trình huấn luyện mô hình. Điều chuẩn là một kỹ thuật nhằm kiểm soát độ phức tạp của mô hình, tránh hiện tượng quá khớp (overfitting) khi mô hình học thuộc nhiều của dữ liệu huấn luyện.

Trong nghiên cứu này, mô hình áp dụng điều chuẩn L2, hay còn gọi là *Ridge regularization*. Cơ chế của điều chuẩn này là giảm giá trị tuyệt đối của các trọng số, khiến các hệ số ước lượng nhỏ hơn nhưng không bị triệt tiêu hoàn toàn. Điều này giúp mô hình ổn định và duy trì khả năng khai thác thông tin của tất cả các biến độc lập. Trong bài toán phát hiện gian lận, dữ liệu thường chứa nhiều đặc trưng có tương quan cao hoặc nhiễu. Việc áp dụng điều chuẩn L2 giúp kiểm soát ảnh hưởng của các đặc trưng này, giảm khả năng mô hình bị lệch về một số biến mạnh, từ đó nâng cao khả năng tổng quát hóa trên tập kiểm định.

Tham số $C=1.0$ – Hệ số nghịch đảo của điều chuẩn:

Tham số *C* điều chỉnh cường độ của điều chuẩn L2. Nó là nghịch đảo của hệ số phạt, tức là khi *C* càng nhỏ, mức điều chuẩn càng mạnh, khiến mô hình bị ràng buộc chặt hơn; ngược lại, khi *C* lớn, điều chuẩn yếu hơn, mô hình linh hoạt hơn nhưng có nguy cơ bị quá khớp.

Trong mô hình này, giá trị $C=1.0$ được lựa chọn, tương ứng với mức điều chuẩn vừa phải. Đây là giá trị cân bằng giữa khả năng khái quát hóa và độ chính xác. Với dữ liệu có tỷ lệ gian lận thấp và nhiễu cao, việc duy trì mức điều chuẩn trung bình giúp mô hình không bị học quá kỹ các mẫu huấn luyện hiếm, đồng thời vẫn đảm bảo phát hiện được các mẫu gian lận tiềm ẩn trong dữ liệu. Trong thực tế, tham số *C* có thể được tinh chỉnh thêm bằng các phương pháp tối ưu siêu tham số (hyperparameter tuning) như Grid Search hoặc Cross Validation, nhưng giá trị mặc định này là khởi đầu hợp lý cho giai đoạn mô hình cơ bản.

Tham số *max_iter*=1000 – Số vòng lặp tối đa khi huấn luyện:

Tham số *max_iter* quy định số vòng lặp tối đa mà thuật toán tối ưu được phép thực hiện trước khi dừng lại. Trong quá trình huấn luyện, mô hình liên tục cập nhật trọng số cho đến khi hội tụ, tức là khi sai số giữa các lần cập nhật liên tiếp nhỏ hơn ngưỡng cho phép.

Nếu giá trị *max_iter* quá thấp, mô hình có thể chưa kịp hội tụ, dẫn đến việc dừng sớm và tạo ra cảnh báo *ConvergenceWarning*. Trong nghiên cứu này, giá trị *max_iter* được đặt là 1000, đảm bảo mô hình có đủ số vòng lặp để đạt được trạng thái hội tụ ổn định, đặc biệt trong trường hợp dữ liệu được chuẩn hóa và có nhiều đặc trưng. Việc lựa chọn giá trị cao hơn mặc định (thường là 100) giúp tránh lỗi huấn luyện chưa đủ thời gian, đồng thời đảm bảo nghiệm tối ưu thu được là đáng tin cậy.

Tham số *random_state*=42 – Hạt giống ngẫu nhiên để tái lập kết quả:

Tham số *random_state* xác định hạt giống ngẫu nhiên cho các quá trình ngẫu nhiên diễn ra trong mô hình, chẳng hạn như khởi tạo trọng số ban đầu hoặc chia tách dữ liệu thành tập huấn luyện và kiểm định.

Việc đặt giá trị cố định cho *random_state* giúp đảm bảo tính tái lập (reproducibility) của kết quả, nghĩa là khi mô hình được huấn luyện nhiều lần trên cùng một tập dữ liệu, kết quả thu được sẽ nhất quán. Trong các nghiên cứu học thuật và đồ án tốt nghiệp, tính tái lập là yếu tố quan trọng để đảm bảo độ tin cậy và khả năng kiểm chứng của kết quả thực nghiệm. Con số 42 được sử dụng như một quy ước phổ biến trong cộng đồng khoa học dữ liệu, không mang ý nghĩa toán học đặc biệt mà chủ yếu nhằm đảm bảo tính nhất quán.

3.1.2. Random Forest

===== Khởi tạo mô hình Random Forest =====

class_weight='balanced' giúp mô hình tự động điều chỉnh trọng số lớp thiểu số

model = *RandomForestClassifier*(

n_estimators=300, # số cây rừng

max_depth=None, # cho phép cây phát triển tự do

min_samples_split=2, # số mẫu tối thiểu để chia node

class_weight='balanced',# cân bằng giữa Fraud/Non-Fraud

```
random_state=42,  
n_jobs=-1          # dùng toàn bộ CPU  
)
```

Giải thích chi tiết các tham số của mô hình Random Forest

Trong nghiên cứu này, mô hình Random Forest (rừng ngẫu nhiên) được sử dụng nhằm giải quyết bài toán phát hiện gian lận trong các giao dịch tài chính. Đây là một phương pháp học máy thuộc nhóm *ensemble learning*, hoạt động bằng cách kết hợp nhiều mô hình con (các cây quyết định) để đưa ra dự đoán cuối cùng. Mỗi cây trong rừng được huấn luyện trên một tập con ngẫu nhiên của dữ liệu và sử dụng một tập ngẫu nhiên các đặc trưng tại mỗi lần chia tách. Cơ chế này giúp giảm phương sai, hạn chế hiện tượng quá khớp (overfitting) và tăng khả năng khái quát hóa của mô hình.

Tham số $n_estimators=300$ – Số lượng cây trong rừng:

Tham số $n_estimators$ quy định số lượng cây quyết định được xây dựng trong mô hình Random Forest. Mỗi cây là một bộ phân loại độc lập, và kết quả cuối cùng được tổng hợp thông qua biểu quyết đa số hoặc trung bình xác suất. Trong mô hình này, giá trị $n_estimators=300$ được lựa chọn để đạt được sự cân bằng giữa hiệu suất và thời gian huấn luyện. Số lượng cây càng lớn thì mô hình càng ổn định, do sai số ngẫu nhiên của từng cây được triệt tiêu dần khi trung bình hóa. Tuy nhiên, khi số lượng cây vượt quá một ngưỡng nhất định, mức cải thiện độ chính xác không còn đáng kể, trong khi chi phí tính toán tăng lên. Với đặc thù dữ liệu phát hiện gian lận có kích thước vừa phải, 300 cây là đủ để mô hình đạt đến điểm hội tụ, đảm bảo độ chính xác cao mà vẫn duy trì tốc độ huấn luyện hợp lý. Việc chọn giá trị này cũng giúp giảm dao động kết quả giữa các lần huấn luyện, tạo ra mô hình ổn định và nhất quán.

Tham số $max_depth=None$ – Độ sâu tối đa của mỗi cây

Tham số max_depth xác định số tầng tối đa mà mỗi cây trong rừng được phép phát triển. Khi $max_depth=None$, cây sẽ tiếp tục chia tách cho đến khi các lá hoàn toàn thuần nhất (chỉ chứa một nhãn duy nhất) hoặc không thể chia tiếp được nữa. Trong nghiên cứu này, việc để $max_depth=None$ cho phép mô hình khai thác triệt để các mối quan hệ phi tuyến tính phức tạp giữa các đặc trưng. Ở mô hình cây đơn lẻ, việc không giới hạn độ sâu có thể dẫn đến quá khớp; tuy nhiên, trong Random Forest, hiện tượng này được khắc phục nhờ cơ chế trung bình hóa giữa nhiều cây độc lập. Mỗi cây chỉ học một phần nhỏ của

dữ liệu, giúp mô hình tổng thể duy trì khả năng khái quát tốt. Do đó, lựa chọn `max_depth=None` là hợp lý, giúp tận dụng tối đa sức mạnh mô hình ensemble mà vẫn hạn chế rủi ro overfitting.

Tham số `min_samples_split=2` – Số mẫu tối thiểu để chia tách một nút

Tham số `min_samples_split` quy định số lượng mẫu tối thiểu cần có trong một nút (node) để được phép tiếp tục chia nhỏ. Mặc định, giá trị này là 2, nghĩa là chỉ cần có từ 2 mẫu trở lên, thuật toán có thể thực hiện phép chia. Trong mô hình này, việc giữ nguyên giá trị `min_samples_split=2` giúp cây có thể học được các chi tiết nhỏ trong dữ liệu. Đặc biệt, trong bài toán phát hiện gian lận, các mẫu gian lận thường rất hiếm và có đặc trưng tinh tế, nên mô hình cần đủ linh hoạt để phát hiện được những mẫu này. Nếu tăng giá trị tham số này (ví dụ lên 5 hoặc 10), cây sẽ bớt chia tách, giúp giảm overfitting nhưng có nguy cơ làm mất khả năng nhận diện các trường hợp gian lận ít gặp. Vì vậy, `min_samples_split=2` được giữ nguyên để đảm bảo mô hình nhạy với các tín hiệu gian lận hiếm, đồng thời vẫn được bảo vệ khỏi quá khớp nhờ cơ chế tổng hợp nhiều cây của Random Forest.

Tham số `class_weight='balanced'` – Cân bằng trọng số giữa các lớp

Tham số `class_weight` điều chỉnh trọng số (weight) của từng lớp trong quá trình huấn luyện, giúp mô hình không bị thiên lệch về lớp chiếm đa số. Khi đặt giá trị là 'balanced', thuật toán tự động gán trọng số tỷ lệ nghịch với tần suất xuất hiện của từng lớp. Trong bài toán phát hiện gian lận, tỷ lệ các giao dịch hợp lệ thường áp đảo tuyệt đối so với giao dịch gian lận. Nếu không có điều chỉnh, mô hình sẽ có xu hướng dự đoán mọi trường hợp là “không gian lận” để tối đa hóa độ chính xác, nhưng kết quả đó lại không có ý nghĩa thực tế.

Việc sử dụng `class_weight='balanced'` giúp mô hình chú trọng hơn đến lớp thiểu số (Fraud), tăng khả năng phát hiện đúng các giao dịch gian lận mà không cần tạo dữ liệu ảo (như kỹ thuật SMOTE). Cách này vừa giữ nguyên cấu trúc dữ liệu gốc, vừa giúp mô hình học được ranh giới phân lớp chính xác hơn. Như vậy, việc lựa chọn `class_weight='balanced'` là giải pháp hợp lý và an toàn để xử lý tình huống dữ liệu mất cân bằng nghiêm trọng.

Tham số `random_state=42` – Hạt giống ngẫu nhiên cho tính tái lập

Tham số `random_state` quy định hạt giống ngẫu nhiên cho các quá trình ngẫu nhiên trong Random Forest, bao gồm chọn mẫu bootstrap và chọn đặc trưng ngẫu nhiên tại mỗi phép chia. Trong học máy, việc sử dụng hạt giống cố

định giúp đảm bảo kết quả có thể tái lập — nghĩa là nếu chạy lại mô hình nhiều lần trên cùng dữ liệu, kết quả thu được sẽ giống nhau. Điều này đặc biệt quan trọng trong các nghiên cứu khoa học và đồ án tốt nghiệp, nhằm đảm bảo tính minh bạch và khả năng kiểm chứng. Giá trị 42 được chọn theo thông lệ phổ biến trong cộng đồng khoa học dữ liệu. Việc gán giá trị cố định này không ảnh hưởng đến bản chất của mô hình, nhưng giúp duy trì sự ổn định và nhất quán trong kết quả thực nghiệm.

Tham số $n_jobs=-1$ – Sử dụng đa lõi CPU để tăng tốc huấn luyện

Tham số n_jobs quy định số lượng lõi xử lý CPU được phép sử dụng trong quá trình huấn luyện mô hình. Khi đặt giá trị là -1, mô hình sẽ tự động tận dụng toàn bộ tài nguyên CPU có sẵn để huấn luyện song song nhiều cây cùng lúc. Vì mỗi cây trong rừng ngẫu nhiên được xây dựng độc lập với nhau, nên quá trình huấn luyện có thể được song song hóa hoàn toàn. Điều này giúp rút ngắn đáng kể thời gian huấn luyện, đặc biệt khi số lượng cây lớn như trong trường hợp này (300 cây). Tham số này không ảnh hưởng đến độ chính xác hay bản chất của mô hình, mà chỉ giúp cải thiện hiệu năng tính toán và tối ưu thời gian thực thi.

3.1.3 XGBOOST

```
model = XGBClassifier(  
    n_estimators=300,      # số lượng cây (boosting rounds)  
    learning_rate=0.05,    # tốc độ học (nhỏ hơn giúp mô hình ổn định hơn)  
    max_depth=5,          # độ sâu tối đa của mỗi cây  
    subsample=0.8,        # lấy ngẫu nhiên 80% mẫu mỗi vòng  
    colsample_bytree=0.8,  # lấy ngẫu nhiên 80% đặc trưng cho mỗi cây  
    random_state=42,  
    scale_pos_weight=scale_pos_weight, # cân bằng giữa lớp 0 và 1  
    eval_metric="logloss", # hàm đánh giá (log loss phù hợp phân loại nhị phân)  
    n_jobs=-1             # dùng toàn bộ CPU để huấn luyện nhanh hơn  
)
```

Phân tích chi tiết các tham số trong mô hình XGBoost

Trong mô hình XGBoost (Extreme Gradient Boosting), việc lựa chọn và điều chỉnh các tham số là yếu tố quyết định hiệu quả học của mô hình. Các siêu tham số dưới đây được lựa chọn dựa trên đặc thù của bài toán phát hiện gian lận (fraud detection) – vốn là dạng dữ liệu mất cân bằng cao, nhiều nhiễu và tín hiệu phân biệt yếu.

$n_estimators = 300$ — Số lượng cây (Boosting Rounds)

Tham số $n_estimators$ quy định số lượng cây quyết định (decision trees) được xây dựng tuần tự trong quá trình boosting. Mỗi cây mới sẽ cố gắng sửa lỗi mà những cây trước đó mắc phải, do đó số lượng cây ảnh hưởng trực tiếp đến độ mạnh (capacity) của mô hình. Việc tăng số lượng cây giúp mô hình biểu diễn phức tạp hơn, khớp tốt hơn với dữ liệu huấn luyện. Tuy nhiên, nếu quá nhiều cây sẽ dẫn đến hiện tượng overfitting, khiến mô hình học theo nhiễu và giảm khả năng khái quát.

Giá trị 300 được chọn vì đây là mức trung bình hợp lý cho các tập dữ liệu vừa và lớn. Nó đủ để mô hình học các mối tương quan phức tạp trong dữ liệu gian lận nhưng không quá lớn để gây quá khớp. Bên cạnh đó, khi kết hợp với $learning_rate$ nhỏ (0.05), mô hình cần nhiều cây để hội tụ — điều này giúp quá trình học diễn ra ổn định và đáng tin cậy hơn. Trong thực tế, tham số này thường được tối ưu thêm bằng early stopping để dừng sớm khi mô hình không còn cải thiện trên tập validation.

$learning_rate = 0.05$ — Tốc độ học (Eta)

Tham số $learning_rate$ (hay còn gọi là *eta*) điều chỉnh mức độ ảnh hưởng của mỗi cây mới khi cộng vào kết quả tổng. Nó đóng vai trò như hệ số giảm tốc, giúp quá trình học tránh bước nhảy quá lớn dẫn đến sai lệch. Giá trị nhỏ khiến mô hình học chậm hơn nhưng ổn định hơn, trong khi giá trị lớn giúp mô hình hội tụ nhanh nhưng dễ rơi vào cực trị cục bộ và overfit.

Giá trị 0.05 được lựa chọn vì đây là mức thấp hơn so với mặc định (0.1), giúp mô hình học từ từ trên dữ liệu nhiễu cao như gian lận. Việc kết hợp $learning_rate$ thấp với $n_estimators$ lớn là chiến lược kinh điển giúp đạt hiệu quả tối ưu và ổn định hơn trong thực tế.

$max_depth = 5$ — Độ sâu tối đa của mỗi cây

max_depth quy định số tầng tối đa của mỗi cây quyết định, tức là mức độ chi tiết khi mô hình chia nhỏ dữ liệu. Cây càng sâu, mô hình càng có khả

năng học được các mối quan hệ phi tuyến phức tạp; tuy nhiên, nếu quá sâu, mô hình dễ học cả nhiễu trong dữ liệu và bị overfitting.

Giá trị 5 là lựa chọn trung gian, thường được dùng cho dữ liệu dạng bảng (tabular data) có nhiều biến như dữ liệu giao dịch tài chính. Mức này giúp cây đủ sâu để nắm bắt các mối tương quan phi tuyến nhưng vẫn đảm bảo mô hình tổng quát tốt khi áp dụng trên dữ liệu mới.

subsample = 0.8 — Tỷ lệ mẫu sử dụng cho mỗi cây (Row Subsampling)

Tham số `subsample` xác định tỷ lệ phần trăm số mẫu được chọn ngẫu nhiên để huấn luyện mỗi cây. Giá trị 0.8 có nghĩa là mỗi cây chỉ được huấn luyện trên 80% dữ liệu huấn luyện ban đầu. Việc này giúp tạo ra tính ngẫu nhiên (stochasticity) giữa các cây, làm giảm tương quan giữa chúng, từ đó giảm overfitting và tăng khả năng khái quát. Tuy nhiên, nếu tỷ lệ này quá thấp, mô hình có thể bị thiếu thông tin, dẫn đến underfitting. Với giá trị 0.8, mô hình vừa đủ đa dạng mà vẫn giữ được lượng thông tin cần thiết — đây là mức được sử dụng phổ biến trong nhiều nghiên cứu thực nghiệm.

colsample_bytree = 0.8 — Tỷ lệ đặc trưng sử dụng cho mỗi cây (Feature Subsampling)

Tham số `colsample_bytree` xác định tỷ lệ số lượng đặc trưng (features) được chọn ngẫu nhiên để huấn luyện mỗi cây. Khi đặt 0.8, mỗi cây chỉ sử dụng 80% số đặc trưng đầu vào. Điều này giúp giảm sự phụ thuộc của mô hình vào một vài đặc trưng mạnh, đồng thời làm tăng tính đa dạng giữa các cây trong mô hình. Nếu tỷ lệ quá thấp, mô hình có thể bỏ qua các biến quan trọng, còn nếu quá cao, các cây sẽ trở nên tương đồng nhau, làm giảm hiệu quả boosting. Giá trị 0.8 được xem là cân bằng giữa hai yếu tố này, đảm bảo mỗi cây vẫn học được phần lớn thông tin nhưng vẫn có sự ngẫu nhiên cần thiết để giảm overfitting.

random_state = 42 — Hạt giống ngẫu nhiên để tái lập (Reproducibility)

`random_state` là hạt giống ngẫu nhiên (seed) được sử dụng để đảm bảo kết quả của các thao tác ngẫu nhiên (chọn mẫu, đặc trưng, khởi tạo trọng số, v.v.) có thể tái lập được. Việc cố định seed đảm bảo rằng khi chạy lại mô hình nhiều lần, kết quả sẽ giống nhau, giúp quá trình thực nghiệm trở nên đáng tin cậy. Con số 42 được sử dụng phổ biến trong cộng đồng khoa học dữ liệu như một giá trị mặc định quen thuộc.

scale_pos_weight = (tỷ lệ mẫu âm/dương) — Trọng số cho lớp dương (Fraud)

Tham số `scale_pos_weight` được thiết kế đặc biệt cho các bài toán mất cân bằng nhãn. Nó quy định trọng số mà mô hình gán cho lớp dương (ở đây là *Fraud*).

Cách tính thông thường:

$$\text{scale_pos_weight} = \frac{\text{số mẫu lớp 0 (Non-Fraud)}}{\text{số mẫu lớp 1 (Fraud)}}$$

Tham số này giúp mô hình coi việc dự đoán sai lớp Fraud là nghiêm trọng hơn, từ đó học cách phân biệt rõ hơn các mẫu hiếm. Đây là cách hiệu quả hơn việc tạo dữ liệu giả như SMOTE, đặc biệt với các mô hình dựa trên cây. Nếu đặt giá trị quá cao, mô hình có thể dự đoán quá nhiều mẫu là Fraud, làm tăng False Positive; ngược lại, giá trị quá thấp khiến mô hình bỏ sót các mẫu Fraud thực sự. Vì vậy, việc tính theo tỷ lệ mẫu là điểm khởi đầu hợp lý và có thể tinh chỉnh thêm dựa trên kết quả thử nghiệm.

eval_metric = "logloss" — Hàm đánh giá trong huấn luyện

Tham số `eval_metric` quy định thước đo được sử dụng để đánh giá hiệu năng mô hình trong quá trình huấn luyện. Trong bài toán này, sử dụng "logloss" (logarithmic loss) – một metric phù hợp cho phân loại nhị phân. Logloss đo độ lệch giữa xác suất dự đoán và nhãn thực tế, phạt mạnh các dự đoán sai lệch lớn. Nó không chỉ đánh giá khả năng phân loại đúng/sai, mà còn phản ánh độ tin cậy của xác suất dự đoán. Trong phát hiện gian lận, việc dự đoán xác suất chính xác là rất quan trọng để điều chỉnh ngưỡng cảnh báo (threshold) theo yêu cầu nghiệp vụ. Vì vậy, logloss là lựa chọn ổn định và hợp lý cho mô hình này.

n_jobs = -1 — Sử dụng CPU song song

Tham số `n_jobs` quy định số lõi CPU được sử dụng để huấn luyện mô hình. Khi đặt giá trị -1, XGBoost sẽ tự động tận dụng toàn bộ tài nguyên CPU khả dụng, giúp tăng tốc quá trình huấn luyện đáng kể mà không ảnh hưởng đến kết quả cuối cùng. Đây là lựa chọn mang tính thực tiễn, đặc biệt hữu ích khi làm việc với các tập dữ liệu lớn như trong bài toán phát hiện gian lận tài chính.

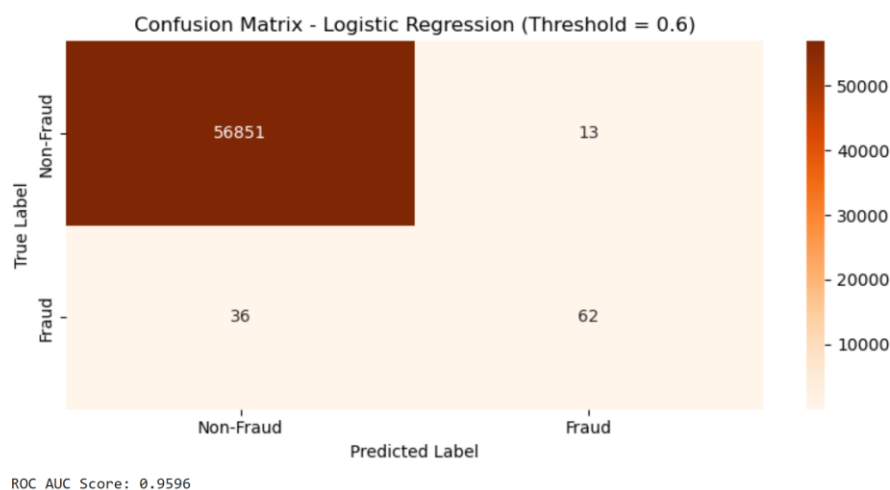
3.2. So sánh và lựa chọn mô hình

3.2.1. Hồi quy Logistic

```
=== Báo cáo phân loại (Logistic Regression cải thiện) ===
              precision    recall  f1-score   support

     0         1.00      1.00      1.00     56864
     1         0.83      0.63      0.72         98

 accuracy          0.91      0.82      0.86     56962
 macro avg          0.91      0.82      0.86     56962
 weighted avg          1.00      1.00      1.00     56962
```



Mô hình Hồi quy Logistic được lựa chọn nhằm đóng vai trò cơ sở (baseline) cho bài toán phân loại gian lận. Với ngưỡng phân loại được điều chỉnh xuống 0.6, mô hình đạt độ chính xác tổng thể (accuracy) gần như tuyệt đối, cùng với chỉ số AUC xấp xỉ 0.96, thể hiện khả năng phân biệt tốt giữa hai lớp “gian lận” và “không gian lận”.

Cụ thể, đối với lớp gian lận (class 1), độ chính xác (precision) đạt 0.83, nghĩa là trong số các giao dịch được mô hình dự đoán là gian lận, có khoảng 83% là chính xác. Độ bao phủ (recall) đạt 0.63, cho thấy mô hình chỉ phát hiện được khoảng 63% các trường hợp gian lận thực tế, bỏ sót 37%. Điểm F1-score đạt 0.72, thể hiện mức cân bằng trung bình giữa khả năng phát hiện đúng và tránh báo sai.

Kết quả ma trận nhầm lẫn cho thấy mô hình dự đoán đúng 56.851 giao dịch hợp lệ và 62 giao dịch gian lận, đồng thời có 36 trường hợp gian lận bị bỏ sót (false negative) và 13 giao dịch hợp lệ bị nhận định nhầm là gian lận (false positive). Điều này cho thấy mô hình hoạt động ổn định và đáng tin cậy, tuy nhiên vẫn còn hạn chế trong khả năng nhận diện các trường hợp gian lận hiếm gặp. Nguyên nhân có thể do đặc trưng tuyến tính của Hồi quy Logistic khiến mô hình chưa thể nắm bắt đầy đủ mối quan hệ phi tuyến giữa các biến đầu vào.

Tổng thể, Hồi quy Logistic là một mô hình nền tảng phù hợp để so sánh, đặc biệt khi yêu cầu tính dễ hiểu và khả năng triển khai nhanh. Tuy nhiên, để cải thiện độ bao phủ (recall), mô hình cần được hỗ trợ thêm bằng các kỹ thuật tiền xử lý dữ liệu hoặc thay đổi ngưỡng phân loại phù hợp hơn.

3.2.2. Random Forest

```

=== Báo cáo phân loại (Random Forest) ===
      precision    recall  f1-score   support

     0.         1.00      1.00      1.00     56864
     1.         0.97      0.59      0.73        98

 accuracy          1.00      56962
 macro avg         0.98      0.80      0.87      56962
 weighted avg      1.00      1.00      1.00      56962

```



Mô hình Random Forest được huấn luyện với tham số `class_weight='balanced'` nhằm giảm thiểu ảnh hưởng của hiện tượng mất cân bằng dữ liệu. Kết quả cho thấy độ chính xác tổng thể đạt mức rất cao, trong đó precision cho lớp gian lận đạt 0.97, recall đạt 0.59 và F1-score đạt 0.73.

Điều này phản ánh rằng mô hình có khả năng dự đoán chính xác rất cao đối với những giao dịch bị gán nhãn là gian lận — tức là hầu như mọi cảnh báo đều đúng (chỉ có 2 trường hợp false positive). Tuy nhiên, khả năng bao phủ của mô hình còn hạn chế, khi chỉ phát hiện được khoảng 59% các giao dịch gian lận, bỏ sót 41% còn lại.

Kết quả ma trận nhầm lẫn minh họa rõ đặc điểm này: trong 98 giao dịch gian lận thực tế, mô hình chỉ nhận diện đúng 58 trường hợp, đồng thời bỏ sót 40 trường hợp (false negative). Mặc dù tỷ lệ báo sai (false positive) cực thấp, song số lượng gian lận bị bỏ sót là khá lớn.

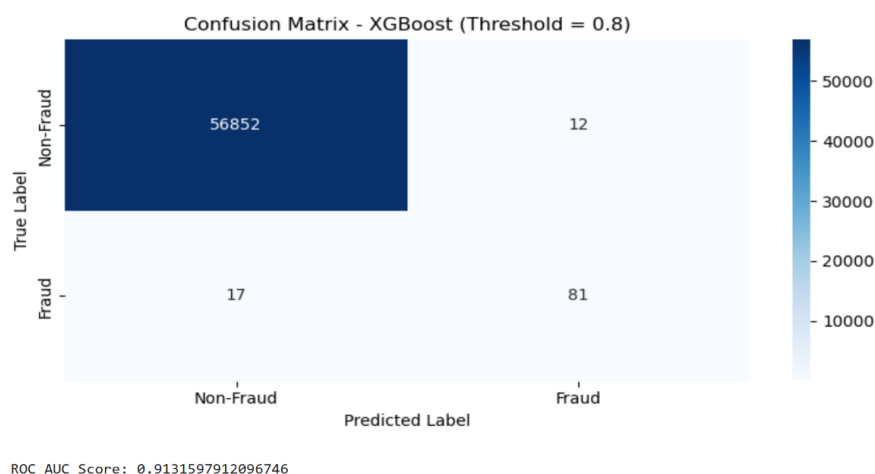
Nguyên nhân của hiện tượng này chủ yếu xuất phát từ ngưỡng phân loại tương đối cao (0.8) cùng đặc điểm của thuật toán Random Forest — vốn có xu hướng tạo ra các xác suất dự đoán tương đối “bảo thủ”. Khi ngưỡng quyết định cao, mô hình ưu tiên tính chắc chắn, dẫn đến giảm đáng kể false positive nhưng tăng false negative.

Nhìn chung, Random Forest là mô hình phù hợp trong các tình huống nghiệp vụ đòi hỏi độ tin cậy cao và chi phí báo sai lớn, ví dụ như khi việc khóa nhầm tài khoản hay cảnh báo sai có thể gây ảnh hưởng nghiêm trọng đến trải nghiệm khách hàng. Tuy nhiên, nếu mục tiêu của hệ thống là phát hiện tối đa các trường hợp gian lận, thì Random Forest chưa phải lựa chọn tối ưu.

3.2.3. XGBoost

=== Báo cáo phân loại ===

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.87	0.83	0.85	98
accuracy			1.00	56962
macro avg	0.94	0.91	0.92	56962
weighted avg	1.00	1.00	1.00	56962



Mô hình XGBoost được huấn luyện với trọng số lớp `scale_pos_weight` nhằm cân bằng dữ liệu và ngăn chặn hiện tượng thiên lệch về lớp chiếm đa số. Với ngưỡng phân loại 0.8, mô hình đạt precision 0.87, recall 0.83 và F1-score 0.85 cho lớp gian lận. Chỉ số AUC đạt khoảng 0.91, phản ánh năng lực phân biệt rất tốt giữa hai nhóm dữ liệu.

Trong ma trận nhầm lẫn, mô hình xác định đúng 81 giao dịch gian lận và chỉ bỏ sót 17 trường hợp. Đồng thời, chỉ có 12 giao dịch hợp lệ bị nhận định nhầm là gian lận. Như vậy, XGBoost đạt được sự cân bằng tốt giữa việc phát

hiện đúng các trường hợp gian lận và hạn chế báo sai, tức là vừa đảm bảo tính hiệu quả vừa duy trì độ tin cậy cao trong dự đoán.

Kết quả trên cho thấy XGBoost có khả năng mô hình hóa mối quan hệ phi tuyến và phức tạp giữa các biến đầu vào, nhờ cơ chế tăng cường dần (boosting) qua nhiều cây quyết định. Thuật toán này hoạt động đặc biệt hiệu quả trong các bài toán có dữ liệu mất cân bằng, bởi khả năng học sâu hơn vào các mẫu thiểu số thông qua việc điều chỉnh trọng số sai số.

Nhìn chung, XGBoost thể hiện hiệu suất vượt trội nhất trong ba mô hình được thử nghiệm, với F1-score cao, recall tốt, và tỷ lệ báo sai thấp. Mô hình này phù hợp với các hệ thống phát hiện gian lận đòi hỏi hiệu quả phát hiện cao và khả năng tổng quát hóa mạnh, đồng thời vẫn giữ được độ ổn định trong quá trình huấn luyện.

3.2.4. So sánh tổng hợp và đánh giá chung

Khi so sánh ba mô hình, có thể nhận thấy rằng:

XGBoost đạt hiệu năng tổng thể tốt nhất, với sự cân bằng tối ưu giữa precision (0.87) và recall (0.83), cùng F1-score cao nhất (0.85).

Hồi quy Logistic có ưu điểm về độ ổn định và khả năng giải thích, song bị hạn chế trong việc phát hiện các mẫu gian lận khó nhận biết (recall thấp hơn).

Random Forest đạt precision rất cao (0.97), nhưng recall chỉ ở mức 0.59, dẫn đến việc bỏ sót nhiều trường hợp gian lận.

Do đó, trong bối cảnh mục tiêu của hệ thống là phát hiện tối đa các giao dịch gian lận, mô hình XGBoost là lựa chọn phù hợp nhất. Nếu yêu cầu đặt nặng vấn đề hạn chế cảnh báo sai, có thể cân nhắc sử dụng Random Forest với ngưỡng phân loại cao hơn. Ngược lại, Hồi quy Logistic có thể được duy trì như một mô hình tham chiếu do tính dễ hiểu và khả năng mở rộng tốt.

3.3. Mô hình XGBoost – Xây dựng, huấn luyện và đánh giá kết quả

3.3.1. Cách XGBoost hoạt động

Về cốt lõi, XGBoost xây dựng một hệ thống mô hình hóa dự đoán bằng cách tuần tự thêm các mô hình đơn giản, thường là cây quyết định, để sửa các lỗi do các mô hình trước đó gây ra. Mỗi cây mới được huấn luyện nhằm giảm

thiếu sai số còn lại (residual error) của mô hình hiện tại. Quá trình này lặp lại nhiều lần cho đến khi mô hình đạt được độ chính xác mong muốn hoặc không còn cải thiện đáng kể. Nhờ cơ chế học tăng cường dần (gradient boosting), XGBoost có khả năng học hỏi hiệu quả từ các sai lầm trong quá khứ, cải thiện dần chất lượng dự đoán.

Nguyên lý học của XGBoost

XGBoost (Extreme Gradient Boosting) dựa trên ý tưởng tối ưu hóa hàm mất mát bằng phương pháp Gradient Descent. Ở mỗi vòng lặp, mô hình tính đạo hàm bậc nhất (gradient) và đạo hàm bậc hai (hessian) của hàm mất mát đối với dự đoán hiện tại, từ đó xác định hướng điều chỉnh trọng số tốt nhất. Mỗi cây quyết định mới được thêm vào sẽ dựa trên các gradient này để giảm thiểu sai số tổng thể.

Điểm khác biệt nổi bật của XGBoost so với các mô hình boosting truyền thống (như AdaBoost) là nó bổ sung cơ chế regularization (L1, L2) để kiểm soát độ phức tạp của từng cây, tránh overfitting, đồng thời sử dụng kỹ thuật song song hóa (parallelization) giúp tăng tốc quá trình huấn luyện đáng kể.

Dữ liệu nền tảng: XGBoost học từ lịch sử

Trong bài toán phát hiện gian lận thẻ tín dụng, XGBoost được huấn luyện trên tập dữ liệu lịch sử chứa các giao dịch thực tế, bao gồm cả hợp lệ (class 0) và gian lận (class 1). Các đặc trưng đầu vào thường bao gồm:

Time: thời gian của giao dịch tính từ mốc đầu tiên, giúp mô hình nhận biết các mẫu hành vi theo thời gian.

Amount: giá trị giao dịch, yếu tố quan trọng để nhận diện giao dịch bất thường.

V1–V28: các thành phần chính (Principal Components) được trích xuất bằng PCA nhằm ẩn danh thông tin gốc nhưng vẫn giữ được cấu trúc quan hệ giữa các biến.

Tập dữ liệu phổ biến từ Kaggle (*creditcard.csv*) gồm 284.807 giao dịch, trong đó chỉ 0.17% là gian lận. Mức độ mất cân bằng này khiến XGBoost cần điều

chỉnh trọng số lớp (thông qua `scale_pos_weight`) để tránh bị thiên lệch về lớp “hợp lệ”. Trong quá trình huấn luyện, mô hình học cách phân biệt các giao dịch dựa trên đặc trưng hành vi, từ đó ước lượng xác suất một giao dịch mới là gian lận.

Cơ chế dự đoán

Sau khi được huấn luyện, XGBoost dự đoán xác suất gian lận cho mỗi giao dịch mới. Xác suất này được so sánh với một ngưỡng phân loại (threshold) — nếu vượt ngưỡng, giao dịch được đánh dấu là “gian lận”, ngược lại là “hợp lệ”. Việc điều chỉnh ngưỡng này ảnh hưởng trực tiếp đến cân bằng giữa độ bao phủ (recall) và độ chính xác (precision):

Ngưỡng thấp → tăng khả năng phát hiện gian lận (recall cao) nhưng dễ báo sai.

Ngưỡng cao → giảm báo sai (precision cao) nhưng có thể bỏ sót các trường hợp thật.

Trong nghiên cứu này, ngưỡng 0.8 được lựa chọn để đạt sự cân bằng hợp lý giữa hai chỉ số này.

Ưu điểm của XGBoost trong phát hiện gian lận

Hiệu quả cao với dữ liệu mất cân bằng: Nhờ cơ chế weighting và boosting, mô hình học tốt các mẫu thiểu số.

Tốc độ huấn luyện nhanh: Hỗ trợ tính toán song song, tối ưu bộ nhớ và cơ chế tree pruning sớm.

Khả năng khái quát mạnh: Có thể phát hiện được các mối quan hệ phi tuyến giữa đặc trưng và nhãn gian lận.

Giải thích được kết quả: Thông qua các công cụ như *Feature Importance* hoặc *SHAP values*, giúp nhà phân tích hiểu đặc trưng nào tác động mạnh nhất đến dự đoán gian lận.

Nhờ các đặc điểm này, XGBoost trở thành mô hình mạnh mẽ và được sử dụng rộng rãi trong lĩnh vực phát hiện gian lận tài chính, nơi yêu cầu vừa chính xác, vừa tốc độ, vừa dễ kiểm soát rủi ro.

3.3.2. Xây dựng và kiểm thử mô hình

Import thư viện cần thiết

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
from xgboost import XGBClassifier
```

Đọc dữ liệu

```
data = pd.read_csv("creditcard.csv")
data.head()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	0
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	0
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	123.50	0
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99	0

5 rows × 31 columns

Kiểm tra sơ bộ dữ liệu

```
print(data.head())
print(data["Class"].value_counts())
```

	Time	V1	V2	V3	V4	V5	V6	V7	\
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	

	V8	V9	...	V21	V22	V23	V24	V25	\
0	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	
1	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	
2	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	
3	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	
4	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	

	V26	V27	V28	Amount	Class
0	-0.189115	0.133558	-0.021053	149.62	0
1	0.125895	-0.008983	0.014724	2.69	0
2	-0.139097	-0.055353	-0.059752	378.66	0
3	-0.221929	0.062723	0.061458	123.50	0
4	0.502292	0.219422	0.215153	69.99	0

[5 rows x 31 columns]

Class

0 284315

1 492

Name: count, dtype: int64

Tách dữ liệu độc lập (X) và phụ thuộc (y), chia tập train, test

`X = data.drop("Class", axis=1)` # X chứa tất cả các cột trừ "Class"

`y = data["Class"]` # y là cột đích cần dự đoán (0 hoặc 1)

test_size=0.2: 20% dữ liệu để test, random_state giúp tái lập kết quả

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

`print("Kích thước tập train:", X_train.shape)`

`print("Kích thước tập test:", X_test.shape)`

Kích thước tập train: (227845, 30)

Kích thước tập test: (56962, 30)

Chuẩn hóa dữ liệu

StandardScaler đưa các đặc trưng về cùng thang đo (mean=0, std=1)

`scaler = StandardScaler()`

`X_train_scaled = scaler.fit_transform(X_train)`

`X_test_scaled = scaler.transform(X_test)`

Khởi tạo và huấn luyện mô hình XGBoost =====

scale_pos_weight giúp cân bằng giữa lớp 0 và lớp 1

= (số mẫu lớp 0 / số mẫu lớp 1)

```
neg, pos = np.bincount(y_train)
```

```
scale_pos_weight = neg / pos
```

```
print("Tỉ lệ scale_pos_weight:", scale_pos_weight)
```

```
model = XGBClassifier(
```

```
    n_estimators=300,      # số lượng cây (boosting rounds)
```

```
    learning_rate=0.05,    # tốc độ học (nhỏ hơn giúp mô hình ổn định hơn)
```

```
    max_depth=5,          # độ sâu tối đa của mỗi cây
```

```
    subsample=0.8,        # lấy ngẫu nhiên 80% mẫu mỗi vòng
```

```
    colsample_bytree=0.8,  # lấy ngẫu nhiên 80% đặc trưng cho mỗi cây
```

```
    random_state=42,
```

```
    scale_pos_weight=scale_pos_weight, # cân bằng giữa lớp 0 và 1
```

```
    eval_metric="logloss", # hàm đánh giá (log loss phù hợp phân loại nhị phân)
```

```
    n_jobs=-1             # dùng toàn bộ CPU để huấn luyện nhanh hơn
```

```
)
```

Tỉ lệ scale_pos_weight: 577.2868020304569

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=0.8, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric='logloss',
               feature_types=None, feature_weights=None, gamma=None,
               grow_policy=None, importance_type=None,
               interaction_constraints=None, learning_rate=0.05, max_bin=None,
               max_cat_threshold=None, max_cat_to_onehot=None,
               max_delta_step=None, max_depth=5, max_leaves=None,
               min_child_weight=None, missing=nan, monotone_constraints=None,
               multi_strategy=None, n_estimators=300, n_jobs=-1,
               num_parallel_tree=None, ...)
```

Dự đoán xác suất thay vì nhãn =====

predict_proba[:, 1] lấy xác suất thuộc lớp 1 (Fraud)

```
y_score = model.predict_proba(X_test_scaled)[:, 1]
```

Chỉnh ngưỡng dự đoán để tối ưu precision hoặc recall =====

```
threshold = 0.8 # bạn có thể thử các giá trị khác: 0.5, 0.7, 0.9,...
y_pred = (y_score >= threshold).astype(int)
Đánh giá mô hình =====
print("\n=== Báo cáo phân loại ===")
print(classification_report(y_test, y_pred, digits=2))
```

Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
labels = ["Non-Fraud", "Fraud"]

plt.figure(figsize=(8, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=labels, yticklabels=labels)
plt.title("Confusion Matrix - XGBoost (Threshold = 0.8)")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.tight_layout()
plt.show()
```

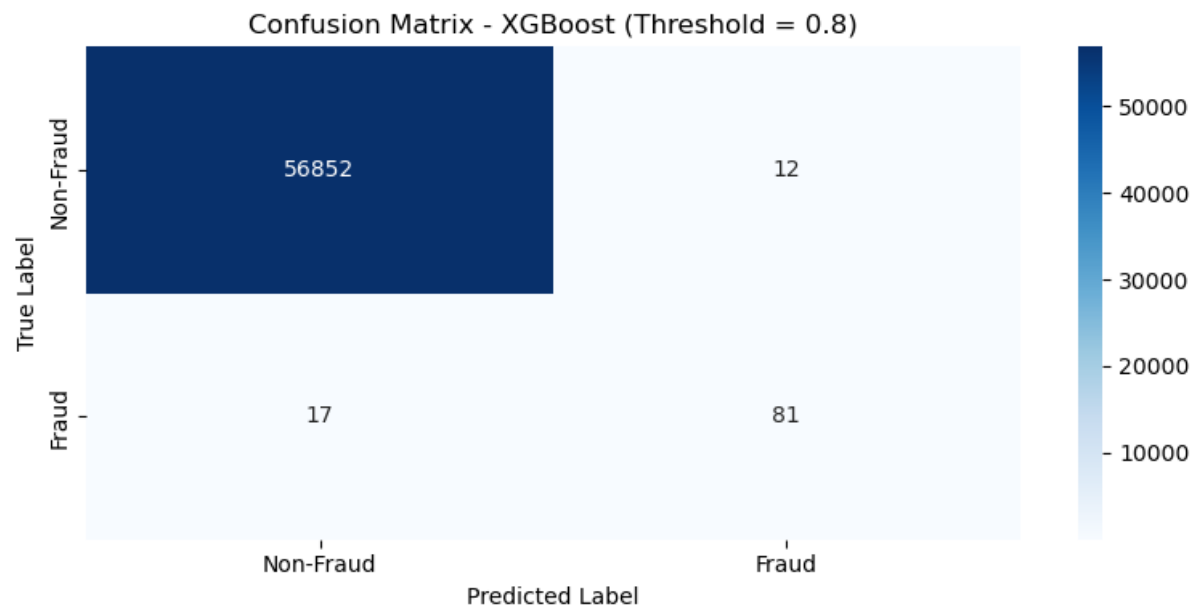
ROC AUC

```
roc = roc_auc_score(y_test, y_pred)
print("\nROC AUC Score:", roc)
```

```
=== Báo cáo phân loại ===
      precision    recall  f1-score   support

     0       1.00      1.00      1.00     56864
     1       0.87      0.83      0.85        98

 accuracy          1.00     56962
 macro avg       0.94      0.91      0.92     56962
weighted avg       1.00      1.00      1.00     56962
```



ROC AUC Score: 0.9131597912096746

Phân tích chi tiết các chỉ số:

Precision 0.87 cho thấy trong các giao dịch bị mô hình dự đoán là gian lận, có tới 87% là chính xác - tức là tỷ lệ báo sai rất thấp.

Recall 0.83 thể hiện khả năng phát hiện đến 83% các trường hợp gian lận thực tế, bỏ sót rất ít.

F1-score 0.85 là cân bằng tốt giữa precision và recall, khẳng định hiệu năng tổng thể cao.

AUC 0.91 phản ánh khả năng mô hình phân biệt mạnh mẽ giữa hai lớp (gian lận và hợp lệ).

TÀI LIỆU THAM KHẢO

1. Kaggle. (2018). *Credit Card Fraud Detection Dataset*.
2. Tạp chí Kinh tế Tài chính Việt Nam. (2024). *Ứng dụng trí tuệ nhân tạo trong phát hiện gian lận tài chính*.
3. Đào, M. A., & Nguyễn, T. T. H. (2018). *Ứng dụng trí tuệ nhân tạo trong hoạt động quản lý rủi ro tín dụng tại các ngân hàng thương mại Việt Nam*. Tạp chí Kinh tế và Phát triển.
4. FPT Digital. (n.d.). *Ứng dụng trí tuệ nhân tạo (AI) phòng tránh rủi ro gian lận trong ngân hàng*.
5. Nguyễn, H. (2019). *Gradient Boosting – Tất tần tật về thuật toán mạnh mẽ nhất trong Machine Learning*. Viblo.
6. Breiman, L. (2001). *Random Forests*. Machine Learning.
7. Mohamed, A. (2023). *Credit Card Fraud Dataset*. Kaggle Notebook.